

Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота № 3

з дисципліни «Розробка мобільних застосунків
під Android»

Тема: «Дослідити способи збереження даних (база даних, файлова
система, тощо) та отримати практичні навички щодо використання
сховищ даних.»

Виконав:
студент групи ІА-31
Сагун Дмитро

Перевірив:
Орленко С.П

Київ 2026

Мета роботи: дослідити створення та взаємодію з компонентом Фрагмент (Fragment) компоненту Діяльність та набуті практичні навички з використання фрагментів для інтерфейсу користувача.

ЗАВДАННЯ

Написати програму під платформу Андроїд, яка має інтерфейс, побудований з декількох фрагментів згідно варіанту. Перший фрагмент представляє з себе форму для введення даних та кнопку підтвердження («ОК»), а інший фрагмент відображає результат взаємодії. Тобто другий фрагмент містить тестове поле з результатом та кнопкою «Cancel» (якщо згідно варіанту така існує, якщо ж за варіантом її немає – можете додати за власним бажанням), яка очищає або приховує (або видаляє) другий фрагмент та очищає форму введення з першого фрагменту. Зверніть увагу, що робота з фрагментами відбувається в рамках однієї Діяльності.

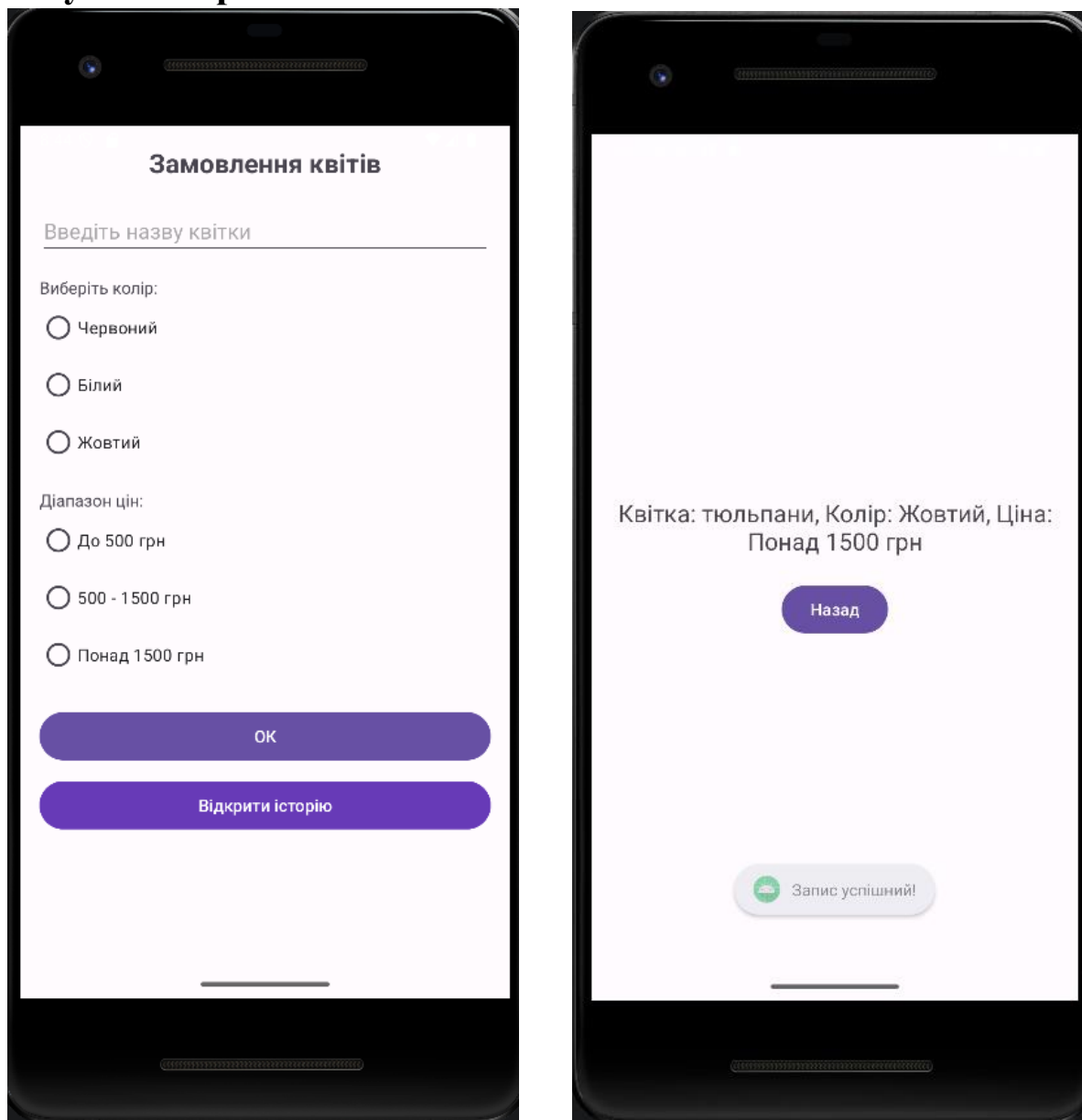
Хід роботи

Варіант	Вікно інтерфейсу
5.	Вікно замовлення квітів містить: текстове поле, дві групи опцій (колір, діапазон цін), тобто радіо-батонів та кнопку «ОК». Вивести інформацію щодо замовлення при натисканні на кнопку «ОК» у деяке текстове поле.

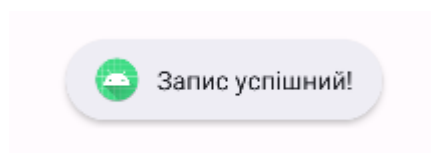
Опис реалізації

- "Для введення даних використано EditText та дві групи RadioGroup (колір та ціна), що забезпечують унікальність вибору."
- "Збереження даних реалізовано у внутрішній текстовий файл flower_orders.txt у режимі MODE_APPEND, що дозволяє додавати нові записи без видалення старих."
- "Навігація між фрагментами здійснюється за допомогою FragmentManager, а перехід до історії — через Intent для запуску HistoryActivity."

Результати роботи



Екран №1: Головна форма



Екран №2: Повідомлення про успіх. Момент, коли з'являється Toast «Запис успішний!».

Відкрити історію

Історія замовлень із файлу:

Квітка: troyanda, Колір: Червоний, Ціна: 500 - 1500 грн

Квітка: troyanda , Колір: Червоний, Ціна: До 500 грн

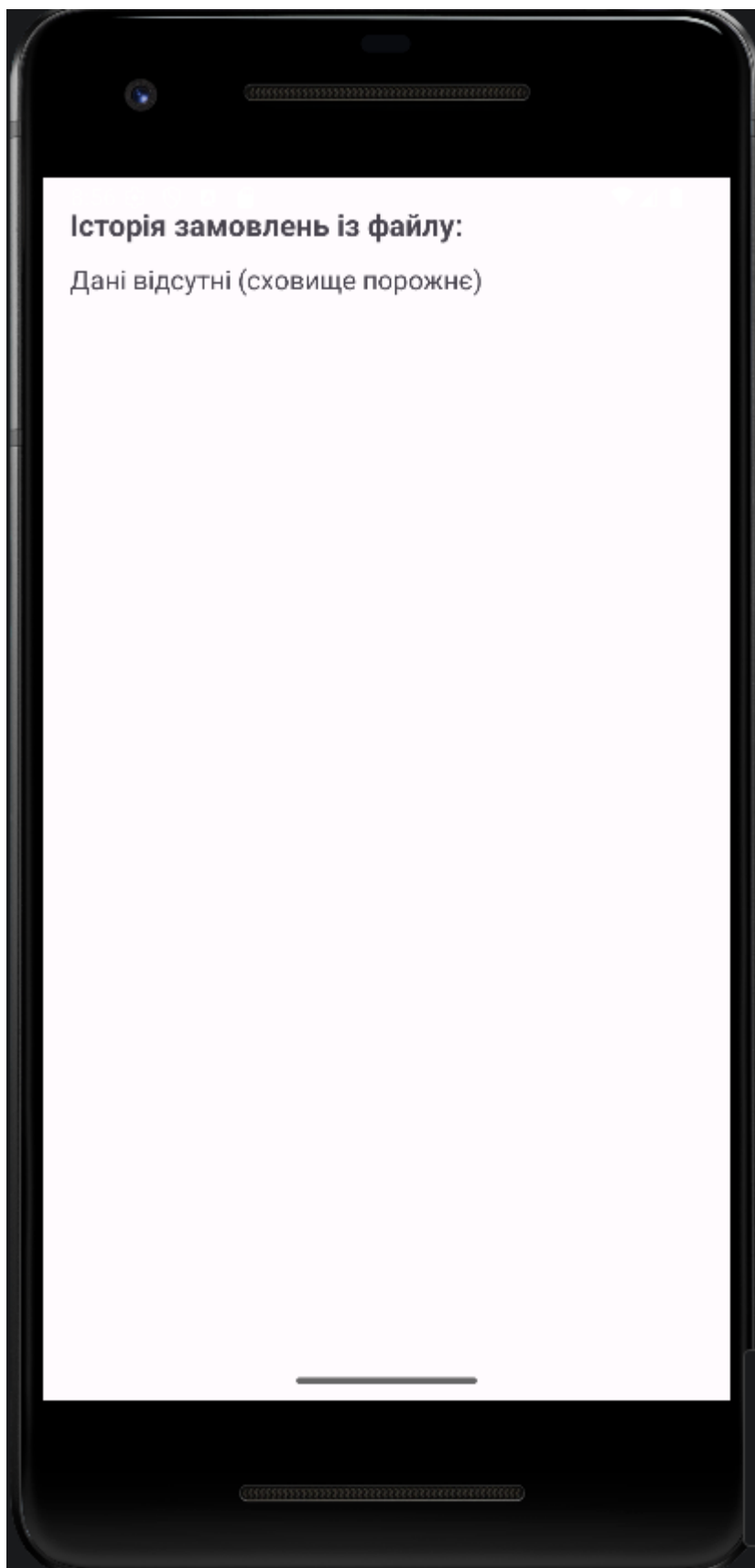
Квітка: тюльпани, Колір: Жовтий, Ціна: Понад 1500 грн

Квітка: соняшник , Колір: Жовтий, Ціна: До 500 грн

Квітка: проліски , Колір: Білий, Ціна: 500 - 1500 грн

Квітка: 1 , Колір: Червоний, Ціна: Понад 1500 грн

Екран №3:Екран історії.



Екран №4: Порожня історія.

ВИСНОВОК

У ході виконання лабораторної роботи №3 було детально досліджено методи збереження даних у платформі Android та отримано практичні навички роботи з внутрішнім сховищем пристрою (**Internal Storage**).

Під час розробки застосунку «Замовлення квітів» було реалізовано:

1. **Персистентність даних:** На відміну від попередніх робіт, де дані існували лише під час роботи програми, у цій роботі реалізовано механізм збереження інформації у текстовий файл `flower_orders.txt`. Використання режиму `Context.MODE_APPEND` дозволило організувати ведення історії замовлень без перезапису попередніх даних.
2. **Обробка виняткових ситуацій:** Реалізовано перевірку наявності файлу перед зчитуванням. Використання блоків `try-catch` дозволило уникнути аварійного завершення програми (Crash) у разі відсутності даних, замінюючи помилку інформативним повідомленням для користувача («Дані відсутні»).
3. **Взаємодія компонентів:** Закріплено навички роботи з **Shared ViewModel** для передачі поточного замовлення та використання **Intent** для переходу між різними Activity (`MainActivity` та `HistoryActivity`).
4. **Користувацький інтерфейс:** Інтерфейс було адаптовано під сенсорне керування з використанням груп радіокнопок (`RadioGroup`), що забезпечує коректний вибір параметрів замовлення (колір, ціна) згідно з документацією Android.

Загальний результат: Створено стабільний застосунок з багаторівневою структурою (Activity -> Fragment -> File System), що відповідає сучасним вимогам до мобільного ПЗ. Отриманий досвід роботи з файловою системою є базою для подальшого вивчення складніших баз даних, таких як SQLite та Room.

Данило, ось розгорнуті та технічно грамотні відповіді на всі 10 контрольних питань. Вони ідеально підійдуть для твого звіту у Word і допоможуть впевнено відповісти викладачу на захисті.

Відповіді на контрольні запитання

1. Робота з налаштуваннями (ключ-значення) В Android для зберігання простих налаштувань (наприклад, ім'я користувача, вибрана тема) використовується **SharedPreferences (або сучасний **Jetpack DataStore**).**

- **Як це працює:** Дані зберігаються у XML-файлі у внутрішній пам'яті додатка. Кожен запис має унікальний текстовий ключ та значення (String, Int, Boolean тощо).
- **Процес:** Для запису використовується спеціальний об'єкт `Editor`. Після внесення змін необхідно викликати `apply()` (асинхронно) або `commit()` (синхронно).

2. Типи сховищ файлів та причини їх використання

- **Internal Storage (Внутрішнє):** Приватне сховище додатка. Файли доступні лише вашій програмі й видаляються разом із нею. Використовується для конфіденційних даних або невеликих файлів (як наш `flower_orders.txt`).

- **External Storage (Зовнішнє):** Спільне сховище (SD-карта або спільні папки). Файли можуть бути доступні іншим додаткам та користувачеві через провідник. Використовується для великих медіа-файлів (фото, відео).

3. Процес роботи з файлами та файловою системою

Робота базується на стандартних класах Java/Kotlin IO:

- **Запис:** Використовується метод `openFileOutput(name, mode)`, який повертає `FileOutputStream`. Режим `MODE_APPEND` дозволяє дописувати дані в кінець файлу.
- **Читання:** Метод `openFileInput(name)` повертає `FileInputStream`. Для зручного зчитування тексту зазвичай використовується `bufferedReader()`.
- **Файлова система:** Додаток працює у своєму "пісочнику" за шляхом `/data/data/[назва_пакета]/files`.

4. Робота з SQLite за допомогою SQLiteOpenHelper

`SQLiteOpenHelper` — це базовий клас для створення та керування версіями локальної БД.

- **Процес:** Потрібно створити власний клас, успадкувати його від `SQLiteOpenHelper` та реалізувати методи `onCreate()` (створення таблиць) та `onUpgrade()` (оновлення структури).
- **Переваги:** Повний контроль над SQL-запитами, не потребує зовнішніх бібліотек.
- **Недоліки:** Велика кількість шаблонного коду (boilerplate), відсутність перевірки запитів під час компіляції (легко припуститися помилки в назві стовпця).

5. Робота з БД за допомогою бібліотеки Room

`Room` — це високорівнева обгортка (ORM) над SQLite від Google.

- **Як працює:** Використовуються три компоненти: **Entity** (опис таблиці через клас), **DAO** (інтерфейс з методами запитів) та **Database** (точка доступу).
- **Переваги:** Перевірка SQL-запитів на етапі компіляції, підтримка `LiveData` та `Coroutines`, набагато менше коду.
- **Недоліки:** Необхідність вивчення анотацій, складніше налаштування для дуже простих задач.

6. Характеристики екранів мобільних пристроїв

- **Розмір діагоналі:** Вимірюється в дюймах.
- **Роздільна здатність:** Кількість пікселів (напр., 1080x2400).
- **Щільність пікселів (DPI/PPI):** Кількість точок на дюйм. Чим вище PPI, тим чіткіше зображення.
- **Співвідношення сторін:** Наприклад, 18:9 або 20:9.

7. Технології екранів та їх відмінності

- **LCD/IPS:** Матриця потребує постійного підсвічування. Кольори природні, але чорний колір виглядає сіруватим. Дешевші у виробництві.
- **OLED/AMOLED:** Кожен піксель світиться окремо. Це дає "нескінченний" контраст, ідеальний чорний колір та економить заряд батареї (чорні пікселі просто вимикаються).

8. Поняття та характеристика сенсорних екранів

Сенсорний екран — це пристрій введення та виведення інформації, що реагує на дотики.

- **Характеристики:** Точність позиціонування, час відгуку, підтримка **Multi-touch** (декілька одночасних дотиків) та рівень прозорості.

9. Загальна класифікація сенсорних екранів

- **Резистивні:** Реагують на фізичне натискання (будь-яким предметом). Дешеві, але недовговічні (зараз майже не використовуються в смартфонах).
- **Ємнісні:** Реагують на дотик пальця (зміну електричного поля). Надійні, підтримують жести, але не працюють у звичайних рукавичках.
- **Інфрачервоні/Хвильові:** Використовуються переважно в платіжних терміналах.

10. Рекомендації щодо розробки інтерфейсів для сенсорних екранів

- **Розмір елементів:** Кнопки повинні бути не менше **48x48 dp**, щоб у них було легко влучити пальцем.
- **Відступи:** Між клікабельними елементами має бути достатньо простору для запобігання помилкових натискань.
- **Жести:** Використання звичних жестів (swipe, pinch-to-zoom).
- **Зворотний зв'язок:** Візуальна зміна стану кнопки при натисканні (як у твоїх `RadioButton`).